

NDAT 6. hafta

Sistem Tasarımı

Analizden Tasarıma Geçiş

- **Analiz (Uygulama Alanı):** "Müşteri ne istiyor?" sorusuna odaklanır.
- Gereksinimler belirlenir ve kullanım durumları (use cases) tanımlanır.
- **Tasarım (Çözüm Alanı):** "Sistemi nasıl kuracağız?" sorusuna odaklanır.
- **Sistem Tasarımı:** Sistemi alt bileşenlere ayırma ve mimariyi oluşturma sürecidir.

Örnek: Bir e-ticaret uygulaması analiz edilirken "Kullanıcı ürün satın alabilmeli" denir (Analiz).

Tasarım aşamasında ise bu işlemin hangi veritabanı ile, hangi sunucu mimarisinde ve hangi güvenlik katmanlarıyla yapılacağı planlanır (Tasarım).

Sistem Tasarımı

- Analiz aşaması uygulama alanına odaklanır.
- Uygulama alanı gerçekleştirilecek yazılımın çalışacağı ortamı ifade eder.
- Tasarım aşaması çözüm alanına odaklanır.
- Tasarım aşaması sistem tasarımı ve nesne tasarımı olarak iki bölüme ayrılabilir.
- Sistem tasarımı kapsamında sistem alt bileşenlerine ayrılır. Sistemle ilgili tasarım hedefleri belirlenir ve kısıtlar ihlal edilmeden bu hedeflere ulaşmak için gerekli mimari oluşturulur.

Sistem Tasarımı

Sistem tasarım aşaması;

- Tasarım hedefleri belirlenir.
- Sistemin alt bileşenlerine ayrıştırılır.
- Tasarım hedeflerine ulaşmayı sağlayacak yazılım mimarisi, veri modelleri, erişim denetimi yöntemleri vb belirlenir.

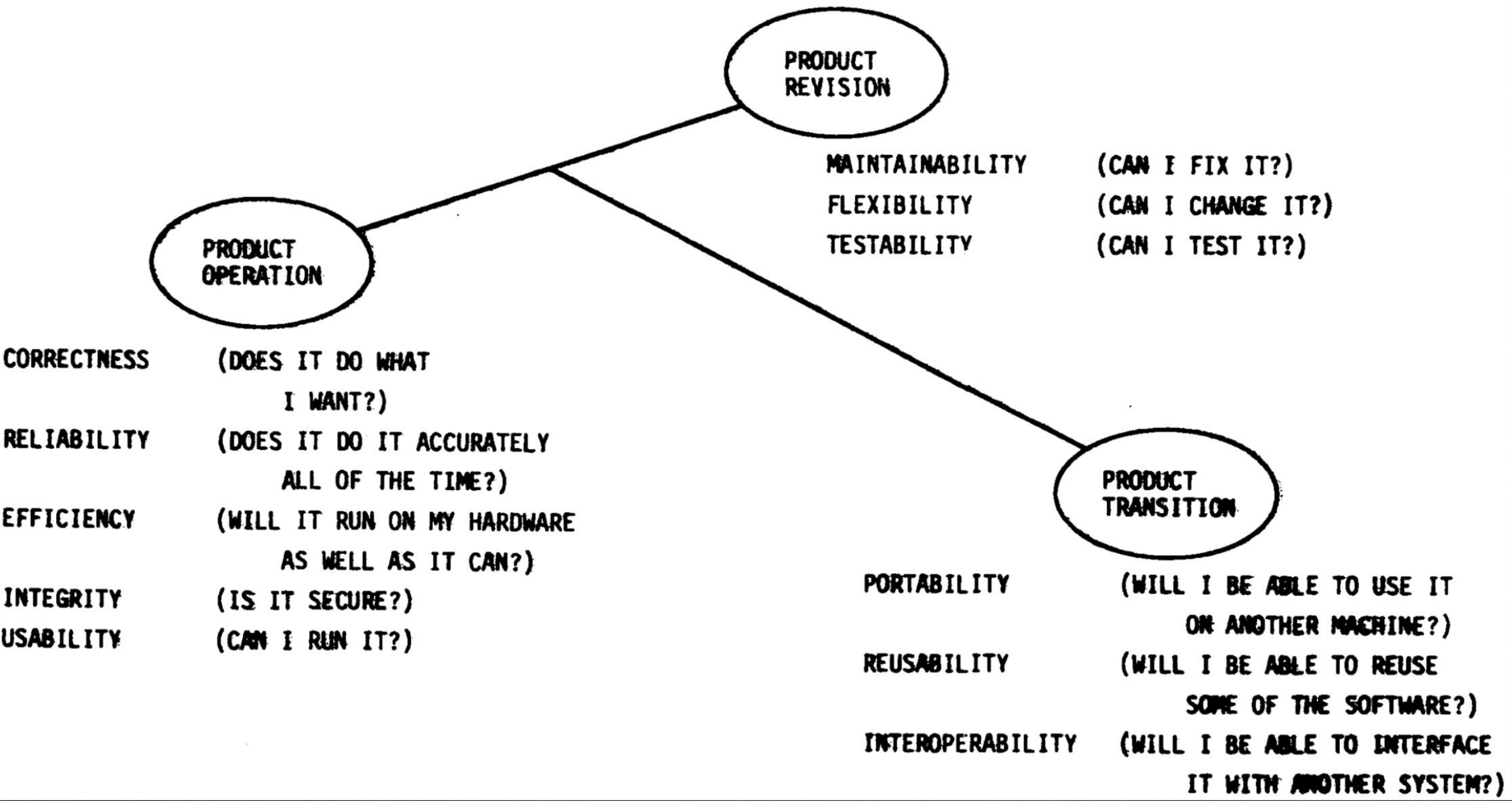
Yazılım Kalitesi ve McCall Üçgeni

Yazılımın kalitesini sadece "çalışması" değil, belirli kriterler belirler:

Ürün Revizyonu: Bakım yapılabilir mi? Test edilebilir mi? Esnek mi?.

Ürün Operasyonu: Doğru çalışıyor mu? Güvenli mi? Verimli mi?.

Ürün Geçişi: Başka platforma taşınabilir mi? Başka sistemle konuşabilir mi?



Kalite Ölçütlerinin Çatışması

- **Güvenlik vs. Verimlilik:** Bir bankacılık uygulamasında her işlemde 3 katmanlı şifreleme (Integrity) isterseniz, işlem hızı (Efficiency) düşer.
- **Taşınabilirlik vs. Hız:** Yazılımın her telefonda çalışmasını (Portability) isterseniz, o donanıma özel hız optimizasyonlarından (Efficiency) vazgeçmiş olursunuz.
- **Güvenlik vs. Performans:** Yazılıma eklenen her şifreleme katmanı, her kimlik doğrulama adımı ve her veri kontrolü, işlemcinin daha fazla mesai yapması ve bellekte daha fazla yer kaplanması demektir. Bankacılık gibi hata payı olmayan sistemlerde hızdan feragat edilip güvenliğe yatırım yapılır.

Kalite Ölçütlerinin Çatışması

- **Esneklik/Taşınabilirlik vs. Verimlilik:** "Her yola gelen" bir araç tasarlamak, o aracın yarış pistindeki saf hızından ödün vermesine neden olur.
- **Yeniden Kullanılabilirlik vs. Hız :** Lego parçaları gibi modüler bir yapı kurmak montajı kolaylaştırır ama yekpare bir gövde kadar sağlam ve hızlı olmayabilir.

Sistemin alt bileşenlerine ayrıştırılması

Alt sistemler, birbirleriyle yakından ilişkili; sınıflar, ilişkiler ve kısıtlardan oluşur.

Alt sistemler sayesinde;

- Problem çözümü kolaylaşır.
- Çok sayıda kişinin aynı projede aynı anda çalışabilmesi sağlanır
- Sistemin anlaşılması, anlatılması kolaylaşır
- Bakım kolaylaşır
- Modülerlik, kod tekrar kullanımı artar.

Analiz aşamasındaki alt bileşenler;

- sınıf
- paket (UML paket şeması)

Tasarım aşamasındaki alt bileşenler için “UML Component” şeması kullanılabilir.

Sistemin alt bileşenlerine ayrıştırılması

- Karmaşıklığı yönetmek için sistem "alt sistemlere" bölünür.
- **High Coherence (Yüksek Bağlılık):** Bir modül sadece kendi işini mükemmel yapmalıdır.
- **Low Coupling (Düşük Bağımlılık):** Modüller birbirine sıkı sıkı bağlı olmamalıdır; birindeki değişiklik diğerini bozmamalıdır.

Görselleştirme Araçları

- UML Paket Diyagramı:** Sınıfları ve bileşenleri klasörler (paketler) gibi gruplar.
- Örnek:** Java'daki `import java.util.ArrayList;` ifadesi bir paket kullanımıdır.
- UML Bileşen (Component) Diyagramı:** Sistemin API'ler üzerinden haberleşen büyük parçalarını gösterir.
- Örnek:** Bir Web Mağazası alt sistemi; Depo (Warehouse) ve Muhasebe (Accounting) alt sistemleriyle konuşur.

UML Paket Diagramı

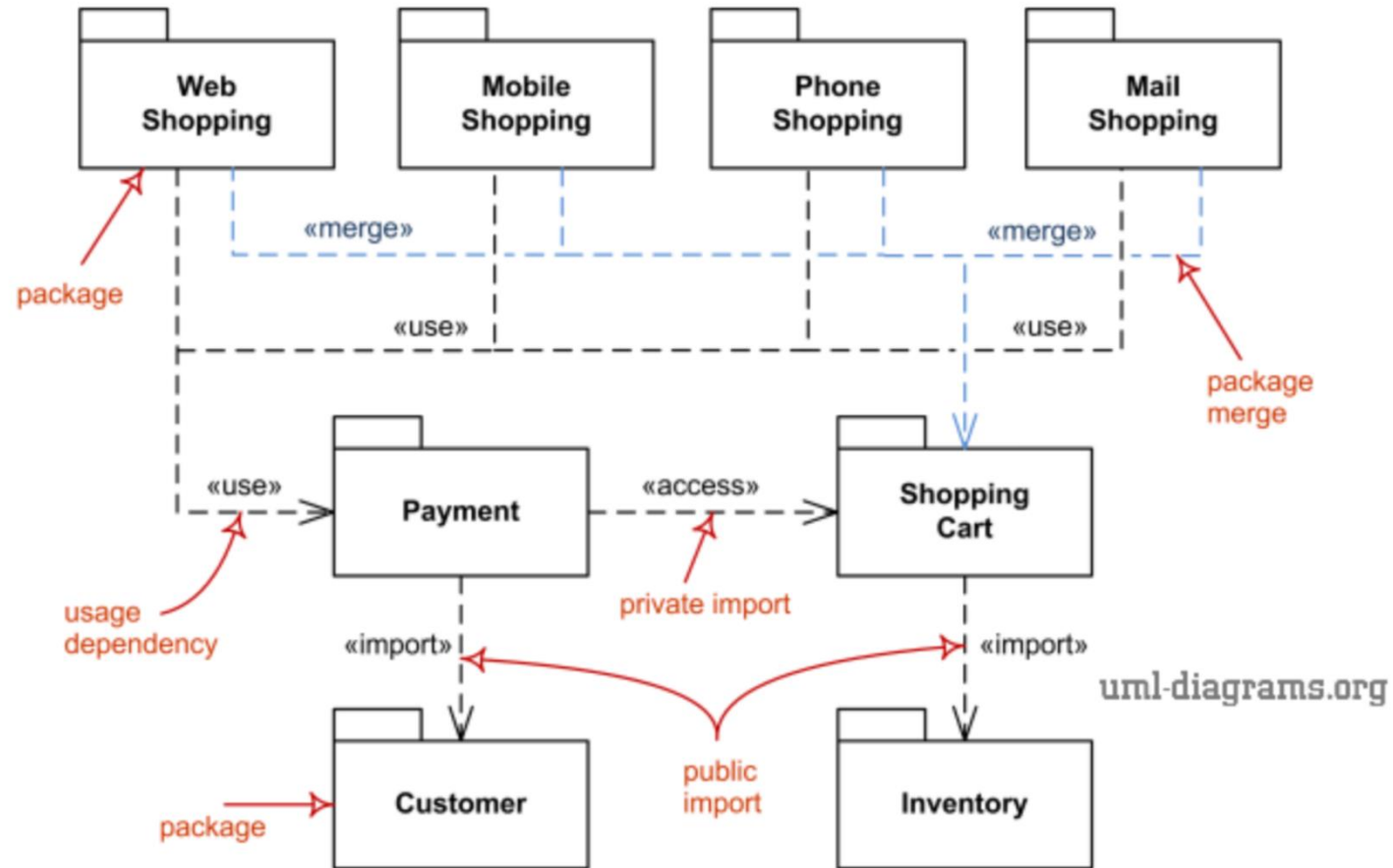
- Yapısal gösterim şekillerindendir.
- Paketleri ve paketler arası ilişkileri göstermek için kullanılır.
- Paketler içerisinde; sınıf, paket, bileşen (component), kısıtlar ve bağımlılıklar olabilir.
- Java platformunda; package ifadesi paket oluşturmak için, import ifadesi ise kullanılacak paketleri içe aktarmak için kullanılır.
- `import java.net.Socket;`
- `import java.io.PrintWriter;`
- `MusteriliSiparis.java`
 - `package cc.ders7.lab;`
 - `import cc.ders5.Musteri;`
 - `import cc.ders6.siparis.Siparis;`

Neden Package?

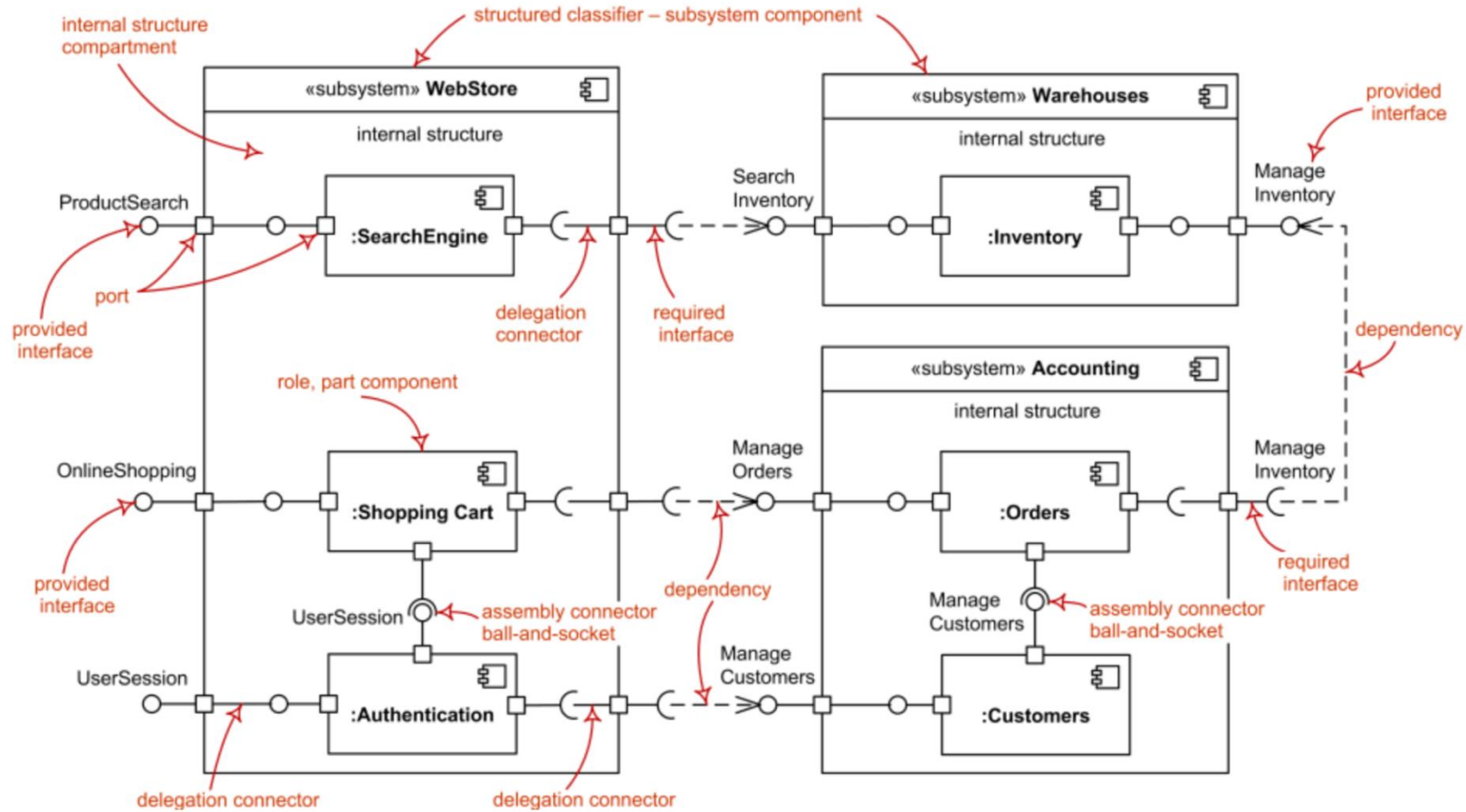
- İsim Çakışmalarını Önlemek (Name Clashing): `com.sirket.proje.User` ve `com.rakip.proje.User`
- Düzen ve Gruplandırma
- Erişim Kontrolü (Encapsulation): Paketler, kodun hangi kısımlarının dışarıya açık olacağını belirlemenizi sağlar. Bazı sınıfları "package-private" (sadece aynı paket içinden erişilebilir) yaparak projenin iç mekanizmasını gizli tutabilir, sadece gerekli API'leri dışarıya açabilirsiniz.
- Yeniden Kullanılabilirlik (Reusability): Bir kez yazdığınız kaliteli bir paketi, başka projelerde de kolayca içe aktararak (import) kullanabilirsiniz. Modern yazılım dünyasındaki NPM (JavaScript), Maven (Java) veya Pip (Python) gibi sistemler tamamen bu mantık üzerine kuruludur.

- `package com.myproject.utils; // Bu sınıfın fiziksel olarak com/myproject/utils klasöründe olması gerekir.`
- `public class Calculator {`
- `// ...`
- `}`

UML Paket Diyagramı



UML “Component” Diyagramı



Modern Yazılım Mimarileri

- **A. Üç Katmanlı Mimari (Three-Tier)**
- **B. Mikro Servis Mimarisi**
- **C. Model-View-Controller (MVC)**

- **İstemci/Sunucu**
- **Servis yönelimli mimari (SOA)**
- **Peer to peer**

Üç katmanlı mimari

Sunum Katmanı

Kullanıcı Arayüzleri
(HTML5, JavaScript, CSS)

Uygulama Katmanı

İş Mantığı
(PHP,Java
Spring,NodeJS...)

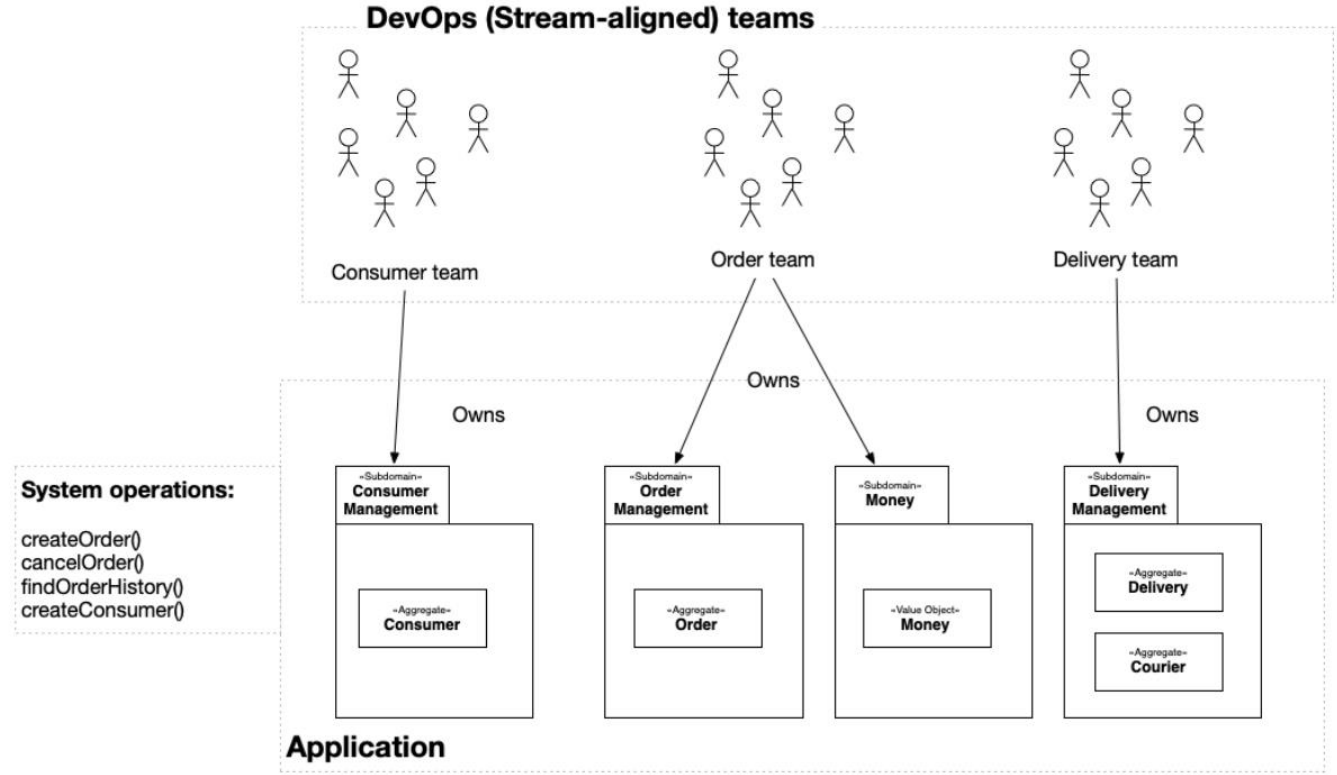
Veri Katmanı

Veri Yönetimi
(PostgreSQL, MongoDB,
MySQL, Oracle, MSSQL,
Cassandra...)

- Sunum, iş mantığı ve veri yönetimi fonksiyonlarının fiziksel olarak ayrıldığı İstemci/Sunucu mimarisidir.
- Modülerlik, bakım, bölümler birbirlerinden bağımsız geliştirilebilir

Mikro Servis Mimarisi

- Uygulamanın çok sayıda küçük ve bağımsız servisten oluşmasıdır.
- **Avantajı:** Her servis (Örn: Ödeme servisi, Stok servisi) bağımsız olarak geliştirilebilir ve test edilebilir.
- **İletişim:** Genellikle REST API üzerinden konuşurlar.



Mikro Servis Mimarisi

Mikroservis Mimarisi

Uygulamayı tek bir parça olarak değil, bir dizi küçük ve bağımsız servis olarak tasarlamaktır.

- Her Servis Bir İş Yapar: Örneğin; Sipariş Servisi, Müşteri Servisi, Envanter Servisi.
- Bağımsız Yaygınlaştırma: Bir servisi güncellemek için diğerlerini durdurmanız gerekmez.
- Teknoloji Çeşitliliği: Bir servis Java ile, diğeri Node.js ile yazılabilir (en uygun araç seçilir)

Mikroservislerin Avantajları

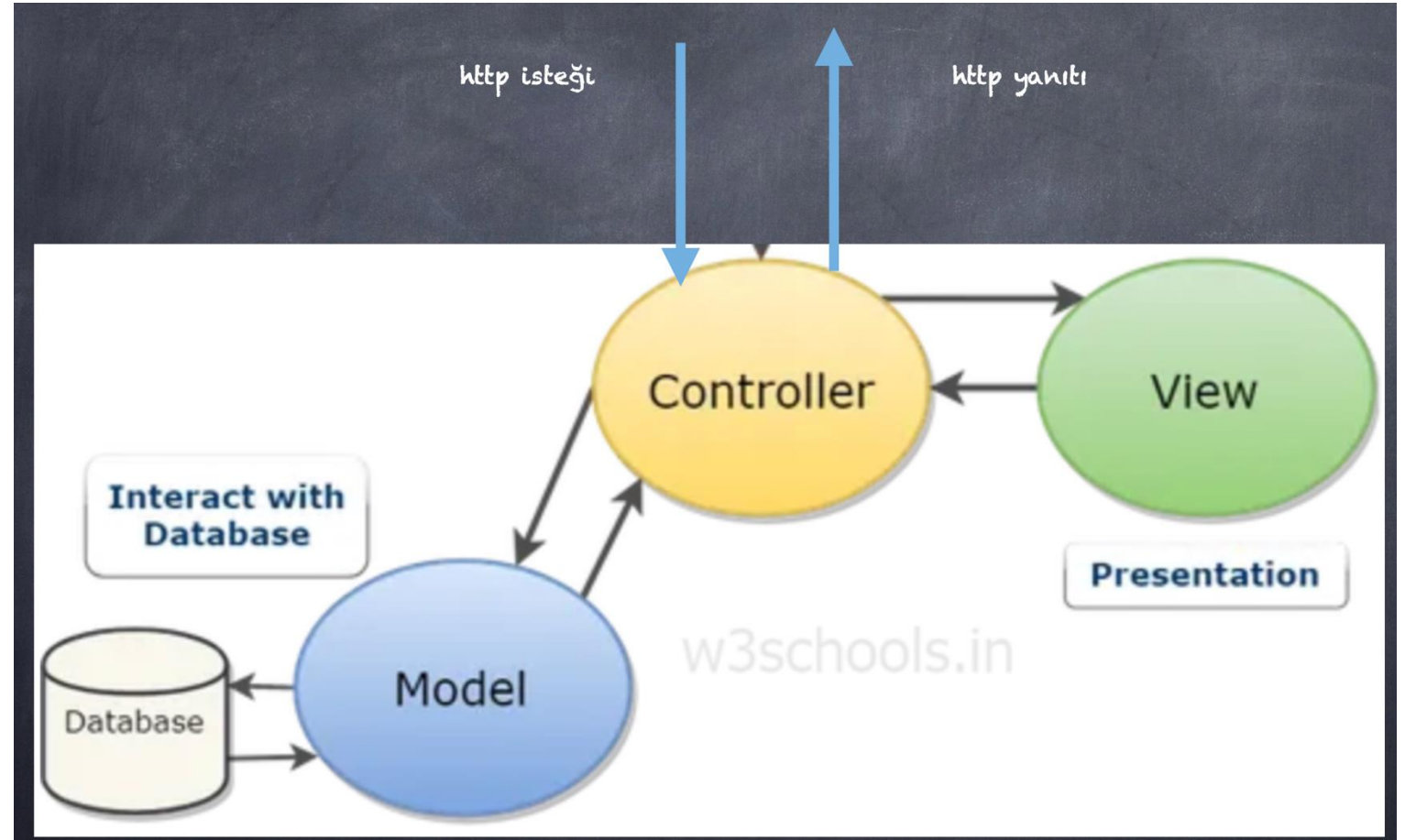
- Hızlı ve Sürekli Teslimat: Değişiklikler anında canlıya alınabilir.
- Kolay Bakım: Her servis küçük olduğu için anlaşılması ve test edilmesi kolaydır.
- Hata İzolasyonu: Bir servisteki çökme (örneğin ödeme servisi), tüm sistemin (örneğin ürün arama) kapanmasına neden olmaz.

Mikroservislerin Zorlukları (Dezavantajları)

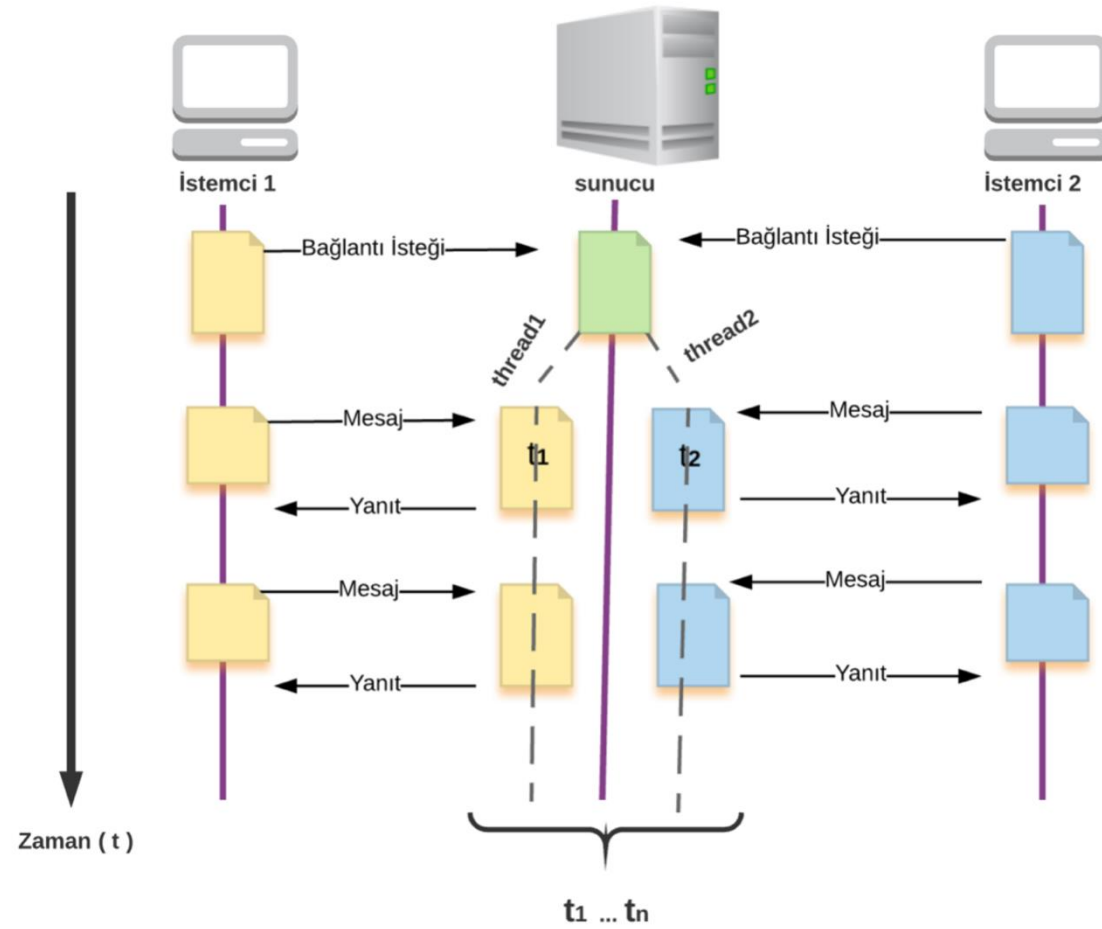
Mikroservisler her sorunu çözen "sihirli bir değnek" değildir; beraberinde yeni karmaşıklıklar getirir:

- Dağıtık Sistem Karmaşıklığı: Servisler arası iletişim (RPC veya mesajlaşma) yönetilmelidir.
- Veri Tutarlılığı: Her servisin kendi veritabanı olduğu için "transaction" (işlem) yönetimi zorlaşır.
- Operasyonel Yük: Bir yerine onlarca servisi izlemek, loglarını tutmak ve yönetmek zordur.

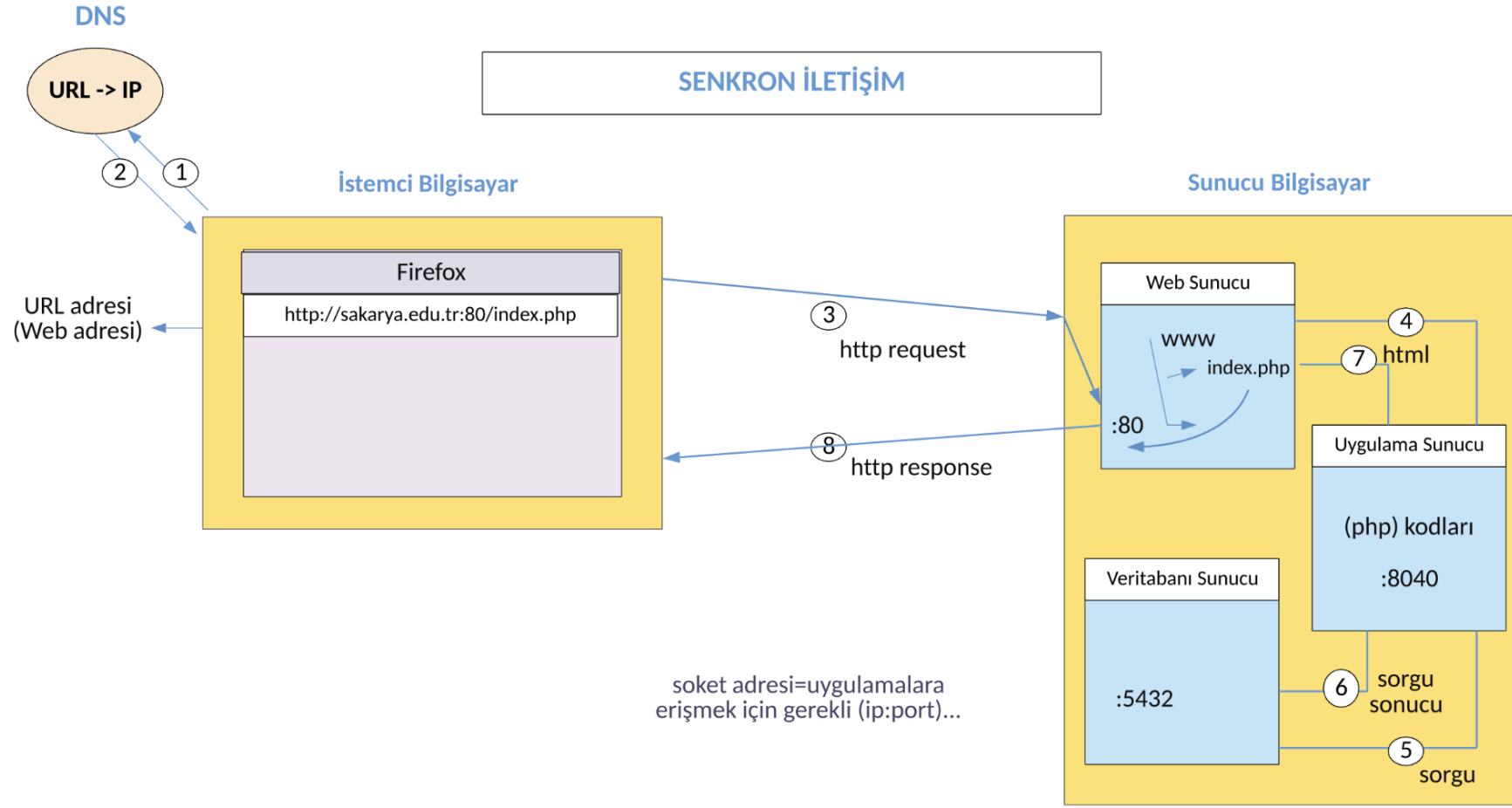
Model View Controller



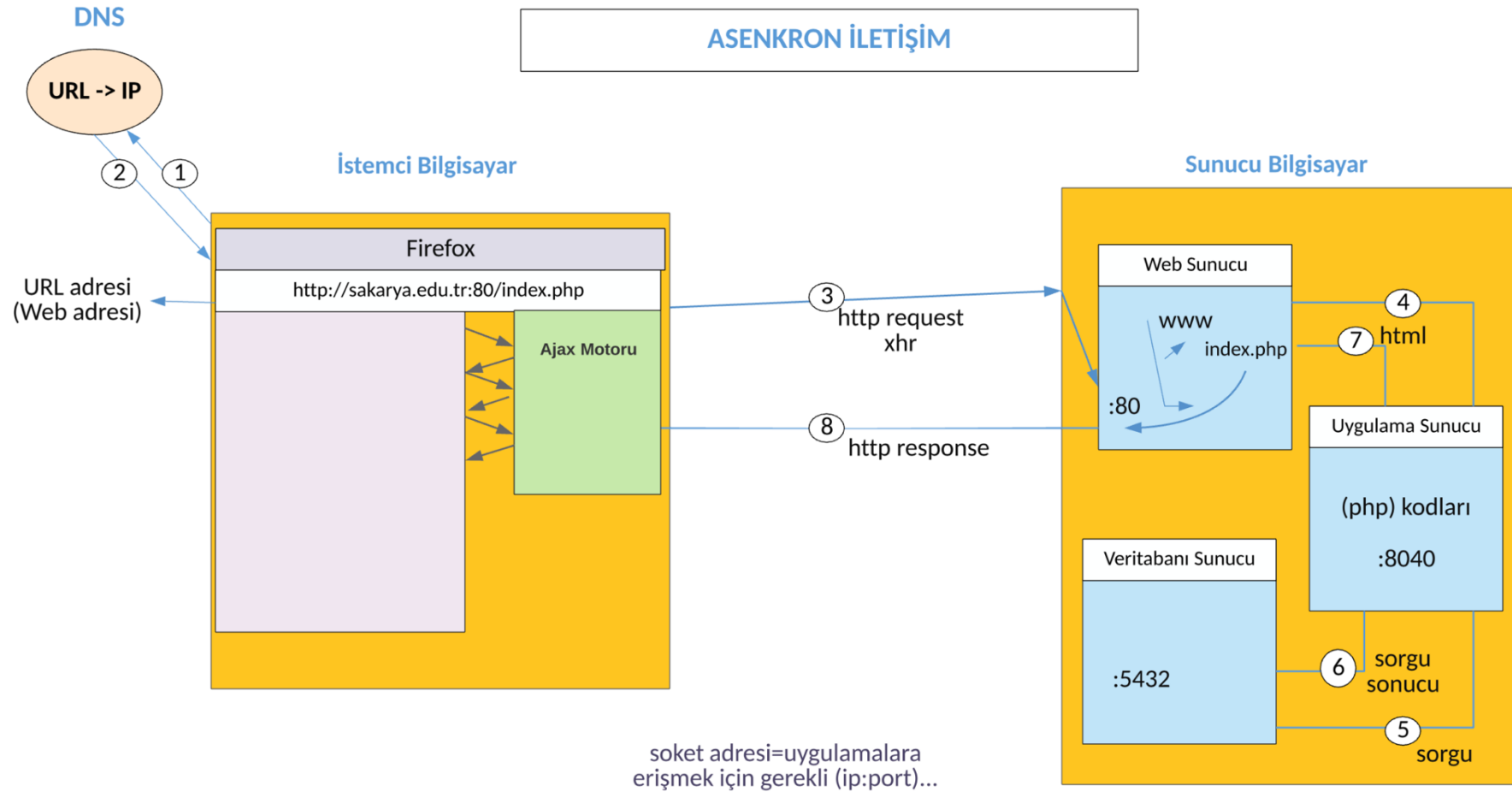
İstemci/Sunucu mimarisi



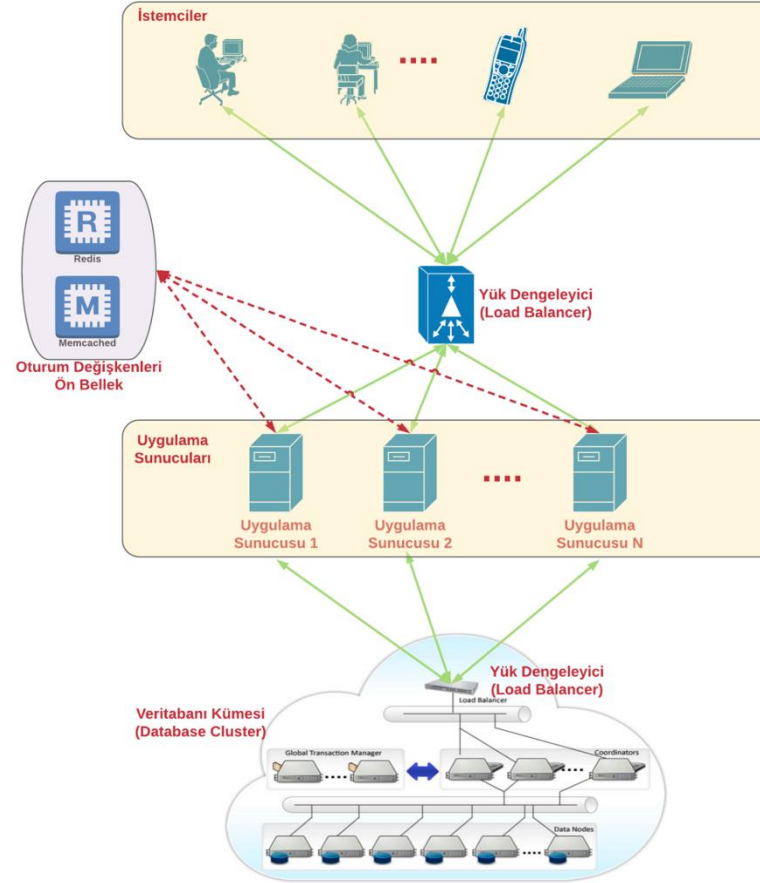
İstemci/Sunucu mimarisi (Web Uygulaması)



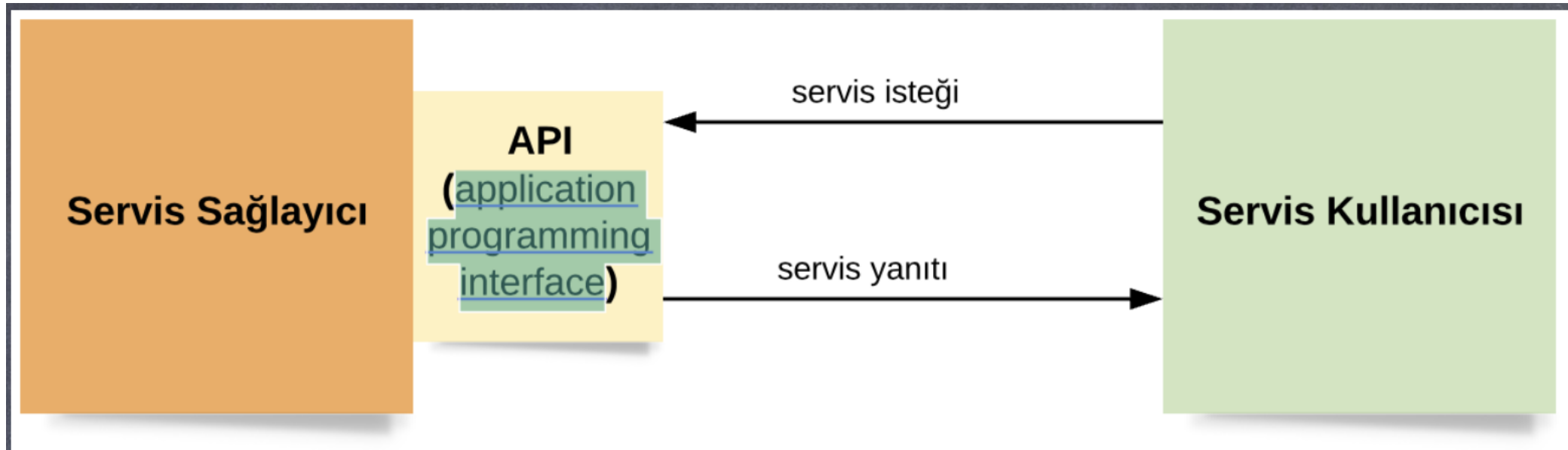
İstemci/Sunucu mimarisi (Web Uygulaması)



İstemci/Sunucu mimarisi (Ölçeklenebilir Web Uygulama Mimarisi)



Servis Yönelimli Analiz (SOA)



SOA-Mikro Servis Yönelimli

1. Servis Yönelimli Mimari (SOA)

SOA, farklı sistemlerin birbirleriyle konuşabilmesini (birlikte çalışabilmesini) sağlamak amacıyla ortaya çıkmıştır.

- Odak Noktası: Kurum içindeki birbirinden farklı devasa sistemleri (örneğin muhasebe yazılımı ile depo yazılımını) birbirine bağlamaktır.
- İletişim: Genellikle ESB (Enterprise Service Bus) adı verilen merkezi bir veri yolu üzerinden haberleşirler.
- Yapı: Servisler genellikle mikro servislere göre daha büyüktür ve birbirlerine bağımlılıkları daha fazla olabilir.
- Protokoller: SOAP, XML-RPC gibi daha ağır ve kuralcı protokoller yaygındır.

2. Mikro Servis Mimarisi

Mikro servisler, tek bir uygulamayı çok sayıda küçük, bağımsız ve kendi başına çalışan parçalara bölerek oluşturur.

- Odak Noktası: Uygulamanın hızla geliştirilmesi, kolayca test edilmesi ve her bir parçanın ihtiyaca göre ayrı ayrı büyütülebilmesidir.
- İletişim: Merkezi bir veri yolu yerine, servisler doğrudan REST API (genellikle JSON desteğiyle), gRPC veya API Gateway üzerinden haberleşir.
- Yapı: Her servis tamamen bağımsızdır. Örneğin, "Ödeme Servisi" hata verse bile "Ürün Arama Servisi" çalışmaya devam edebilir.
- Veritabanı: SOA'da genellikle ortak bir veritabanı kullanılırken, mikro servislerde her servisin kendi özel veritabanı (Account DB, Inventory DB vb.) olması tercih edilir.

Web İletişim Yöntemleri

- **Senkron İletişim:** İstemci istek gönderir ve yanıt gelene kadar bekler.
- **Asenkron İletişim (AJAX):** Sayfa yenilenmeden arka planda veri alışverişi yapılır.
- **Ölçeklenebilirlik:** Yük dengeleyiciler (Load Balancer) ve ön bellek mekanizmaları (Redis, Memcached) kullanılarak sistemin binlerce kişiye hizmet vermesi sağlanır.